ISTA 130

Functions, Types, Math

What’s new?

More work with functions.

Review of types.

Calculations in Python.

Last time

More Turtle graphics.

Functions and code reuse.

The argument/parameter passing process.

Functions

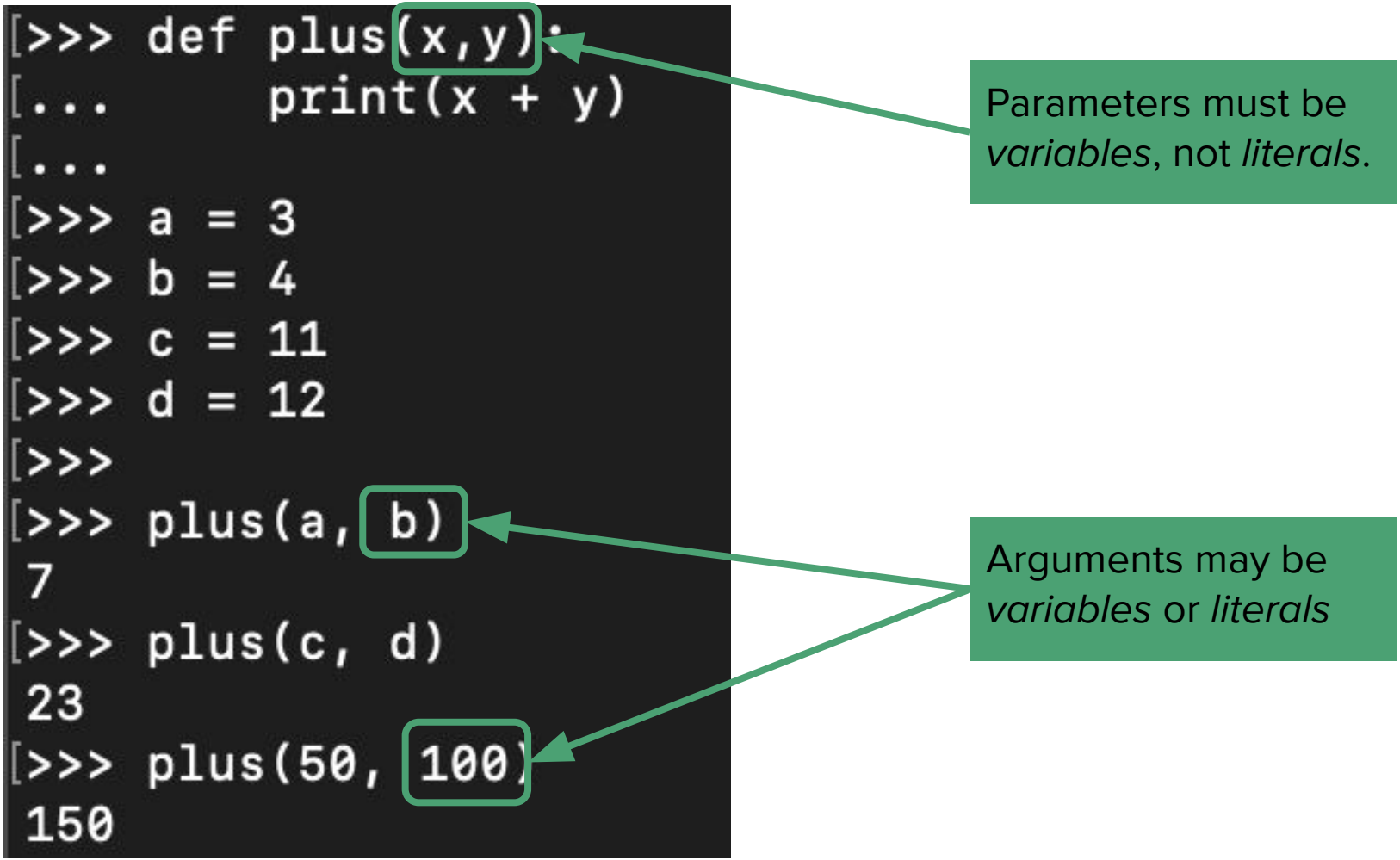
What is a function

➔ A block of code that has a name so that the block can be executed with a single instruction.

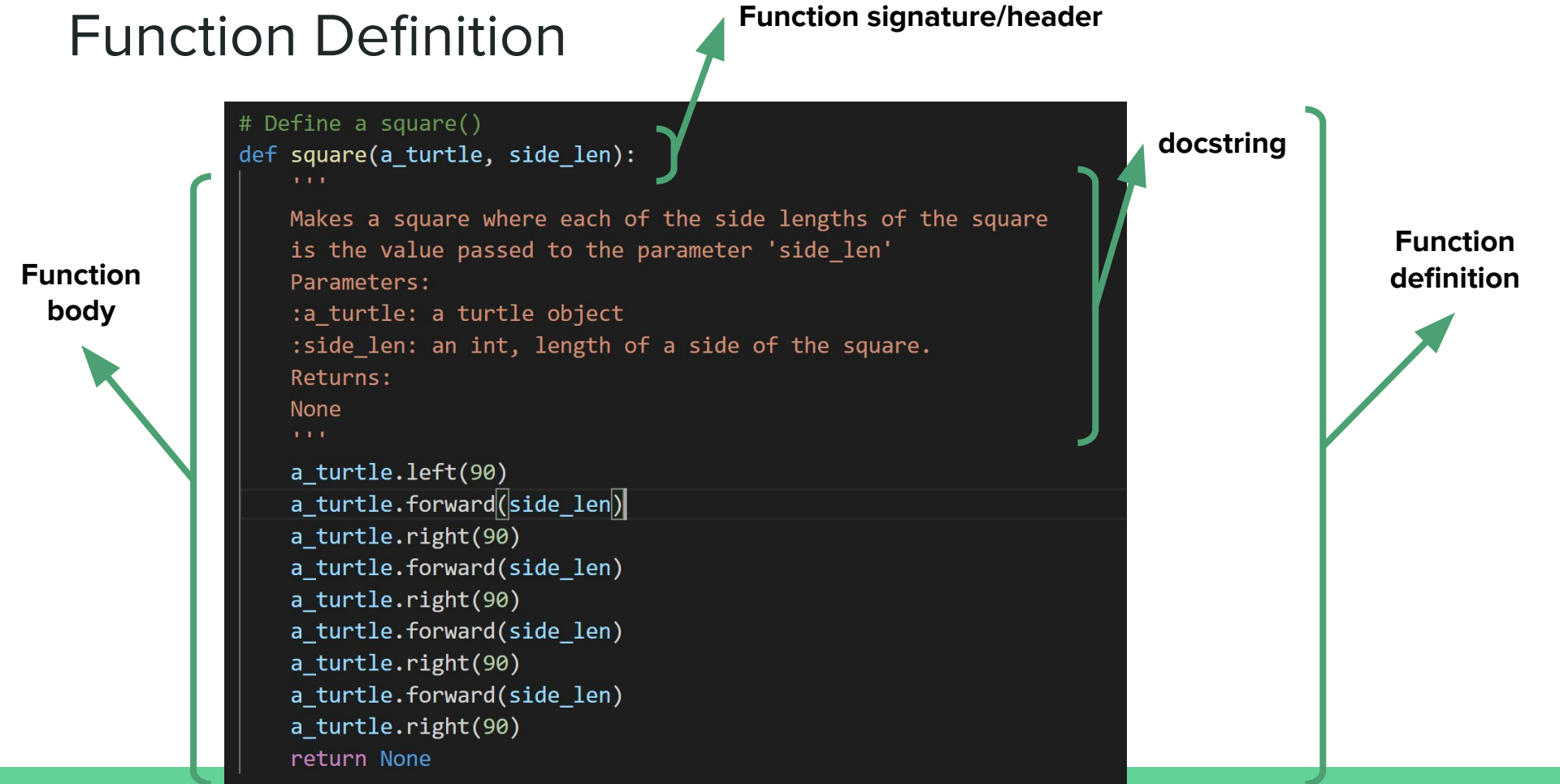
What is the purpose of functions?

➔ Code reuse
➔ Problem decomposition

Functions allow us to write clear, easy-to-maintain code. In other words, good code.


>>> def plus(x,y):
... print(x + y)
...
>>> a = 3
>>> b = 4
>>> c = 11
>>> d = 12
>>>
>>> plus(a, b)
7
>>> plus(c, d)
23
>>> plus(50, 100)
150
Parameters must be *variables*, not *literals*.
Arguments may be *variables* or *literals*

Function Definition



Function Signatures in Hw's

Function signatures/headers (the first line in the function definition) must match the provided specification exactly - same function name, same number of parameters in the same order. Parameter names don't matter, but if the specification (spec, for short) says the first parameter is a turtle, the next an integer, this order must be respected.

The grading scripts make calls to your functions to test them and if your functions don't match the format expected by the grading script, your function won't pass the tests.

Function Behaviors in Hw's

Functions must perform exactly as specified to get full credit - nothing less, nothing more.

Later on, if you are putting **`print`** calls in your code to debug, remember to remove them once they are no longer needed!

Names of Variables, Functions, and Parameters

Begin names with letters, separate words with underscores, use lowercase letters and numbers.

Variable names must be meaningful. Do not use single-letter variable names. Or two-letter names.

Python uses **`lowercase_w_underscores`**, not **`camelCase`** like Java.

https://www.python.org/dev/peps/pep-0008/

Text Editor Syntax Highlighting

Syntax highlighting: your editor knows something about Python and provides helpful coloring.

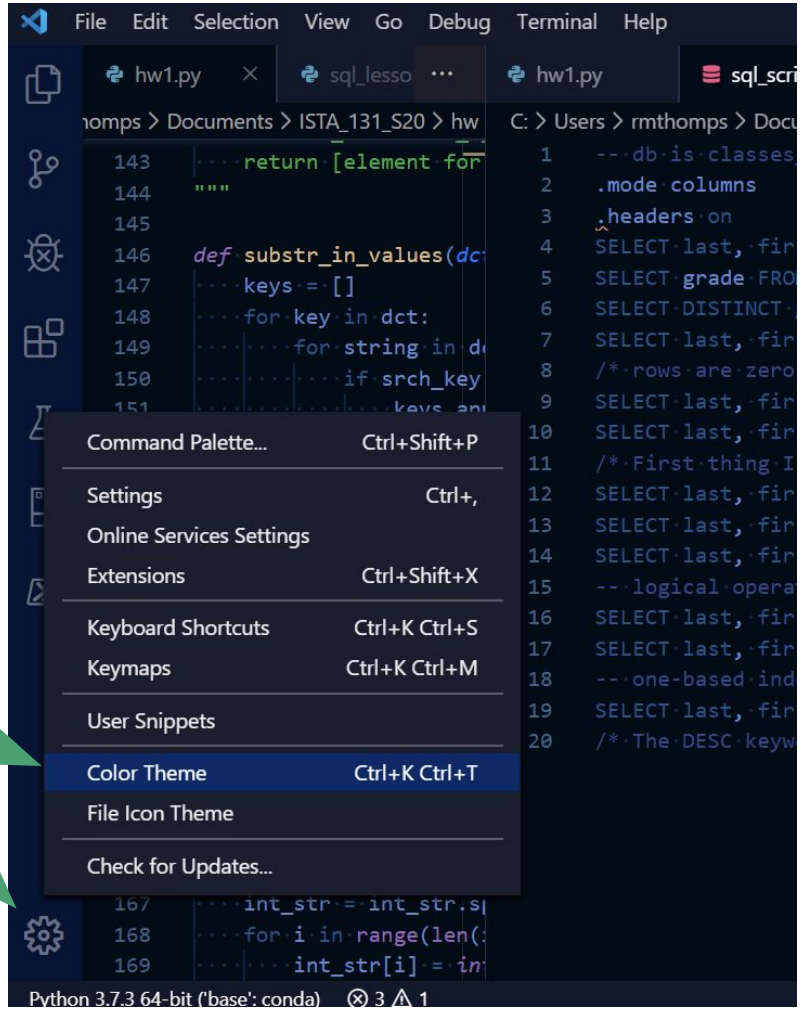
Reserved words: words Python owns and you cannot use as names - these will have a special color that depends on the color scheme.

Parenthesis matching functionality - this is awesome!

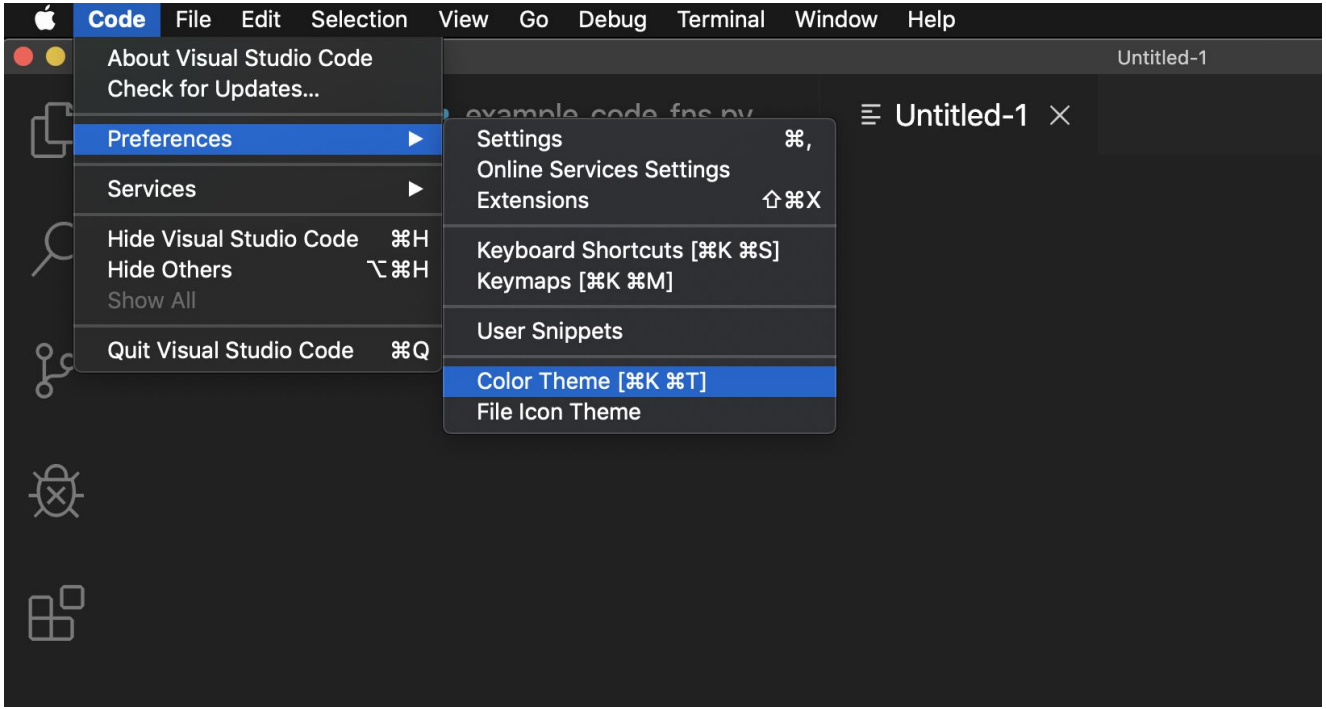
You can pick your color scheme. (see the tutorials)

VSCode: Choose a color scheme (Windows)

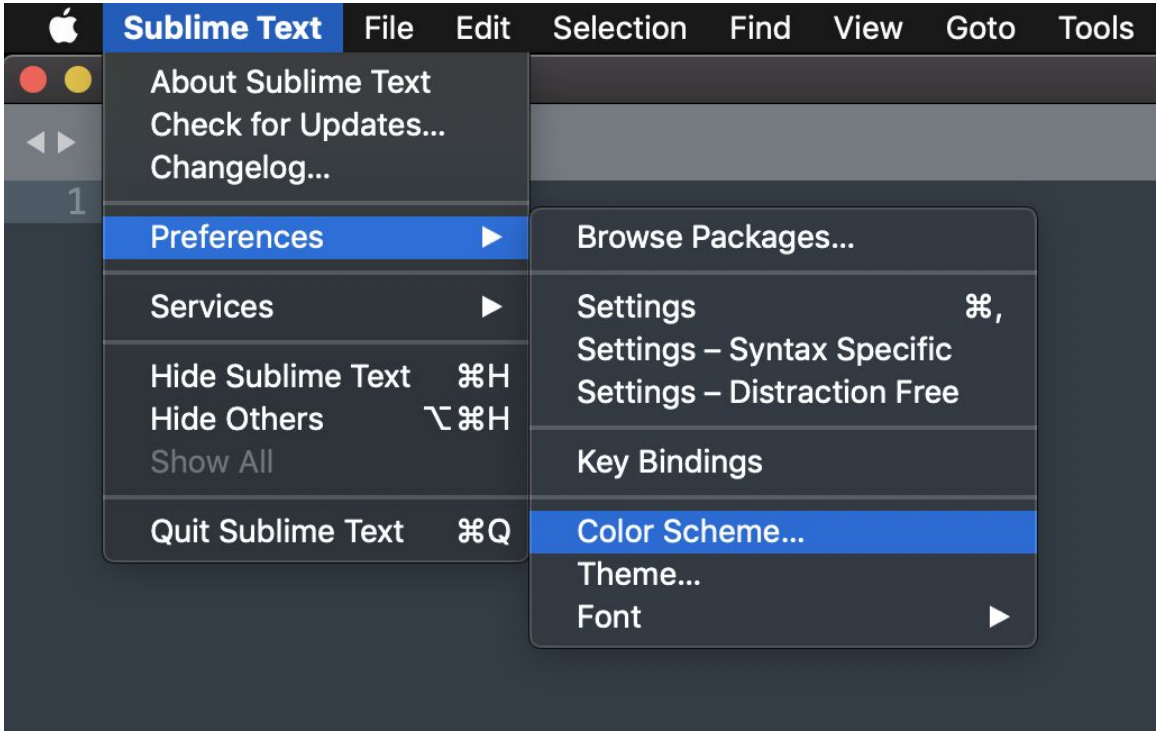
Click the Settings gear icon and select ‘Color Theme’



VSCode: Choose a color scheme (Mac)



Sublime Text: Choose a color scheme (Mac)



Program Layout: organizing your script

Should include, in order:

1. A long documentation string.
2. Any import statements.
3. All function definitions.
4. A comment line we choose to put in for clarity. (`#==========`)
5. The **`main()`** function.





`square()`

Back to our square

Is the `a_turtle.right(90)` call necessary?

Square is drawn without it, but it is much harder to figure out how to do the next thing if the turtle doesn't end up heading in the same direction it started.

```
def square(a_turtle):
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    return None
```

Let’s add to our function repertoire

In this week’s coding tutorial, we’ll draw a house and use an equilateral triangle function and a square function to draw a house.

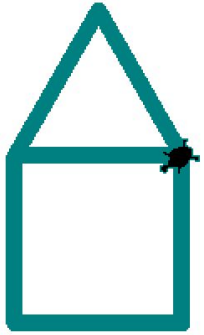
```
def triangle(a_turtle, side_len):

def square(a_turtle, side_len):

def house(a_turtle, width):

    triangle(a_turtle, side_len)

    square(a_turtle, side_len)
```



Let’s add to our function repertoire

In this week’s coding tutorial, we’ll draw a house and use an equilateral triangle () function and a square function to draw a house.

But what if our turtle wants a door for their house? What do we need?

```
def rectangle(???)
```



Multiple parameters

We can have as many parameters as we would like.

Parameters are in the function definition (in the signature) and are separated by commas.

Arguments are passed to functions, also separated by commas, and are matched with the corresponding parameters.

Let’s add to our function repertoire

In this week’s coding tutorial, we’ll draw a house and use an equilateral triangle () function and a square function to draw a house.

But what if our turtle wants a door for their house? What do we need?

```
def rectangle(a_turtle, width, height):

    ...
```



Let’s add to our function repertoire

In this week’s coding tutorial, we’ll draw a house and use an equilateral triangle () function and a square function to draw a house.

But what if our turtle wants a door for their house? What do we need?



```
def main():

    tallulah = turtle.Turtle()

    rectangle(tallulah, 50, 100)
```

Calling the `rectangle` Function



"Hey rectangle, do your thing."

"And use these values this time:"



50 100

Function Receiving a Call

When the function receives a call, it assigns each of the objects/values in the call's argument list a name from its parameter list:

```
def rectangle(a_turtle, width, height):
```

`rectangle` Receiving a Call



"Got it, thanks."

"I'm going to label these objects:"

a_turtle



width

50

height

100

Let’s throw a door on our house

Do we still need **`square()`** if we have **`rectangle()`**?

Now we’ll put our door into **`house()`**.

To walk through the step-by-step process of writing these functions, and making a house, watch the Let’s Draw a House tutorial.

What if we want to make adjustments to argument values?

When values (arguments) are passed into our functions, sometimes we need to modify them in order to achieve our desired result.

To do so, we can perform mathematical operations in Python, and they look very similar to what you might expect.

```
def half(num):
    '''
    This function returns the value passed to 'num'
    divided by 2.
    '''
    return num / 2
```

Review of Types of Values

Python recognizes many different **types** of values:

`int` (short for integer)

➔ `0, 7, -21, 4593211,-101`

➔ Do not put commas in ints (`1000`, not 1,000).

`float`

➔ `3.14159, 2.71828, 5.0, -0.99999`

`str` (short for string)

➔ `"Blue",'salmon',"why is that salmon blue?"`

More types coming

Types

Types are categories of objects that share common properties and behaviors. Like integers. And turtles.

The built-in `type()` function can tell us what kind of object we're looking at:

```
>>> type(snoopy)
<class 'turtle.Turtle'>
>>> type(33)
<class 'int'>
>>> type('hmmmm')
<class 'str'>
>>> type(type)
<class 'type'>
>>>
```

Python Calculates

Order of operations: PEMDAS

- ➔ Parentheses: x * (y + z)
- ➔ Exponentiation: x ** y
- ➔ Multiplication and Division: x * y, x / y
- ➔ Addition and Subtraction: x + y, x - y

You can use parentheses to force the order you want (or just to make your code more readable).

To do

Watch Coding example: Let's Draw a House

Quiz 2 - Function Jargon

Functions Discussion

Hw1